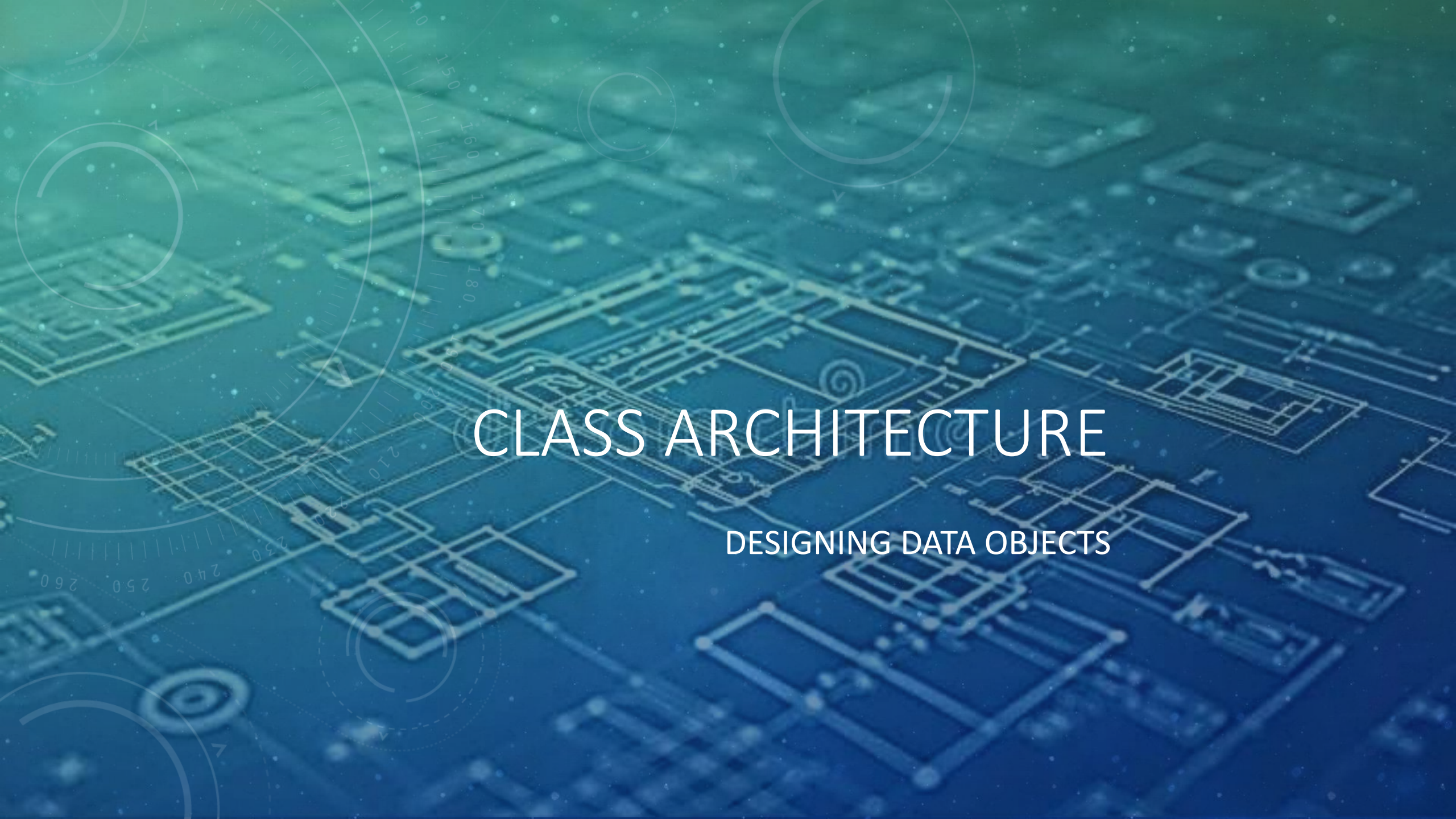


CLASS ARCHITECTURE

DESIGNING DATA OBJECTS

What is Data?

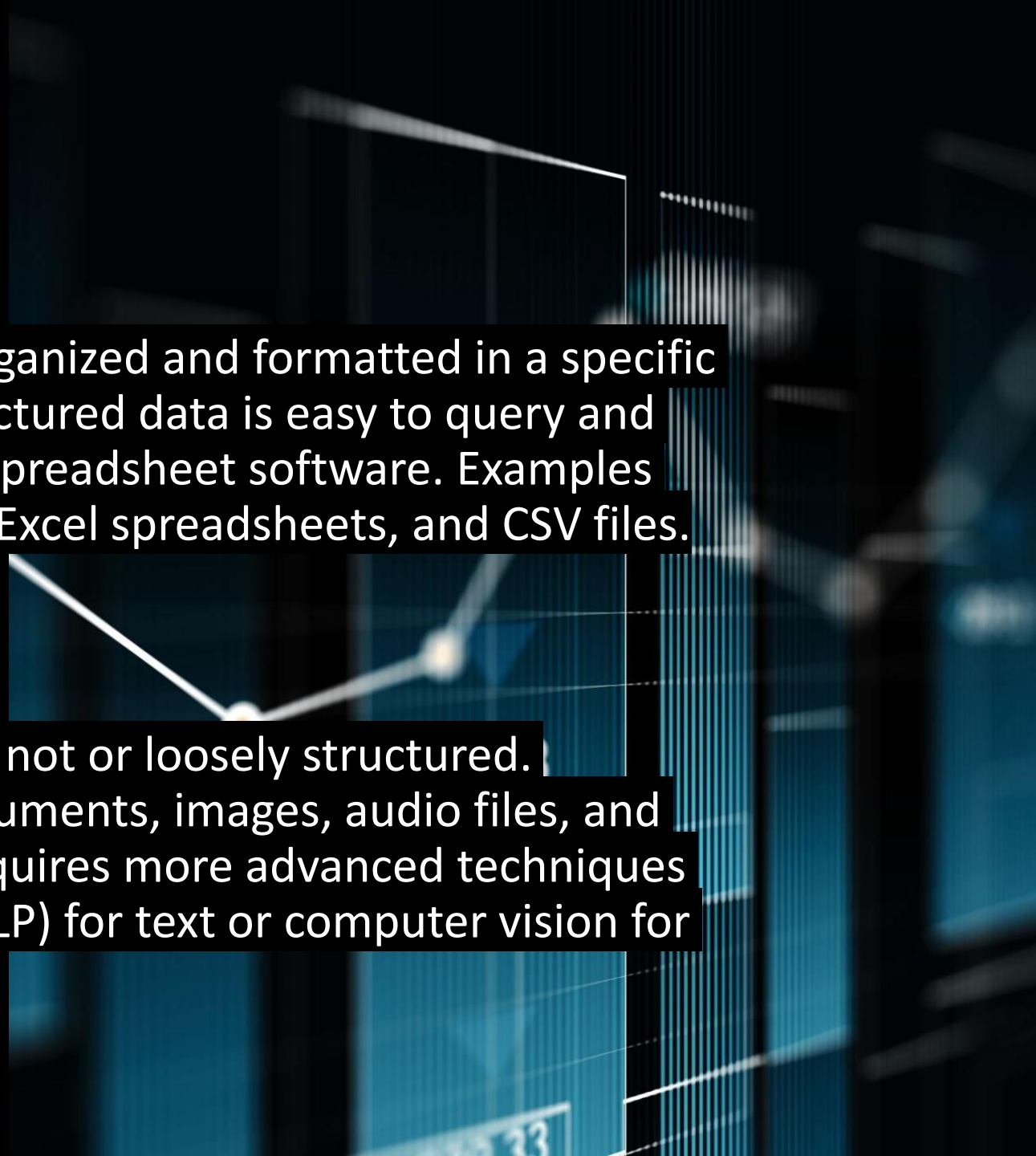
- Data refers to raw facts, information, or statistics that are collected, processed, and stored for various purposes.

- It can take various forms, such as numbers, text, images, audio, and more. Data is the foundation for generating knowledge and insights when analyzed and interpreted.

Types of Data

- **Structured Data:** This type of data is organized and formatted in a specific way, often into rows and columns. Structured data is easy to query and analyze using tools like databases and spreadsheet software. Examples include tables in a relational database, Excel spreadsheets, and CSV files.

- **Unstructured Data:** This type of data is not or loosely structured. Unstructured data can include text documents, images, audio files, and videos. Analyzing unstructured data requires more advanced techniques such as natural language processing (NLP) for text or computer vision for images.

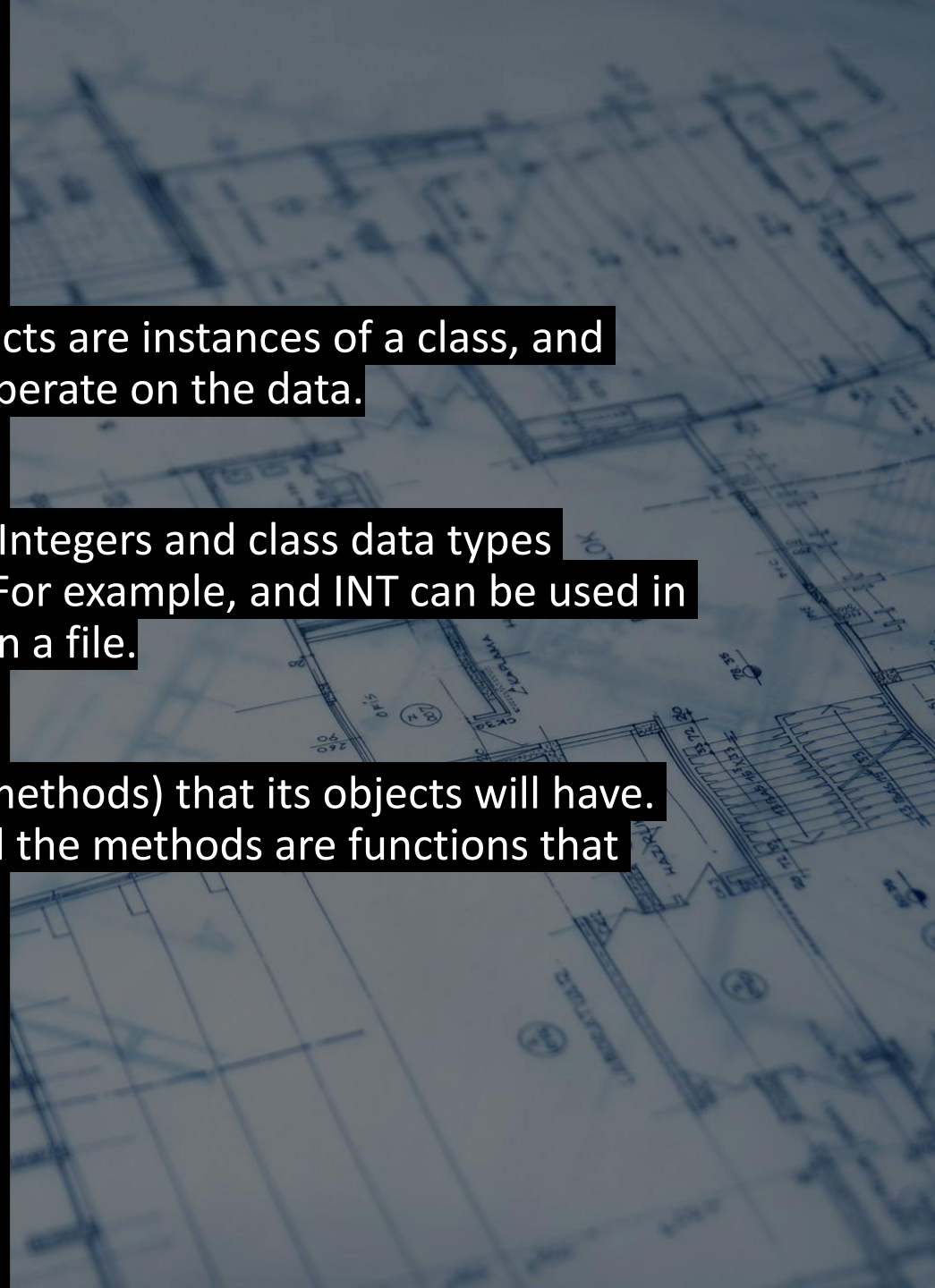


What is a Class?

- A class is a blueprint or template for creating objects. Objects are instances of a class, and they **encapsulate data** and the methods (functions) that operate on the data.

- A class defines a Data Type, just like an INT is a data type. Integers and class data types perform predefined behaviors required by the data type. For example, and INT can be used in mathematical operations and a File Reader object can open a file.

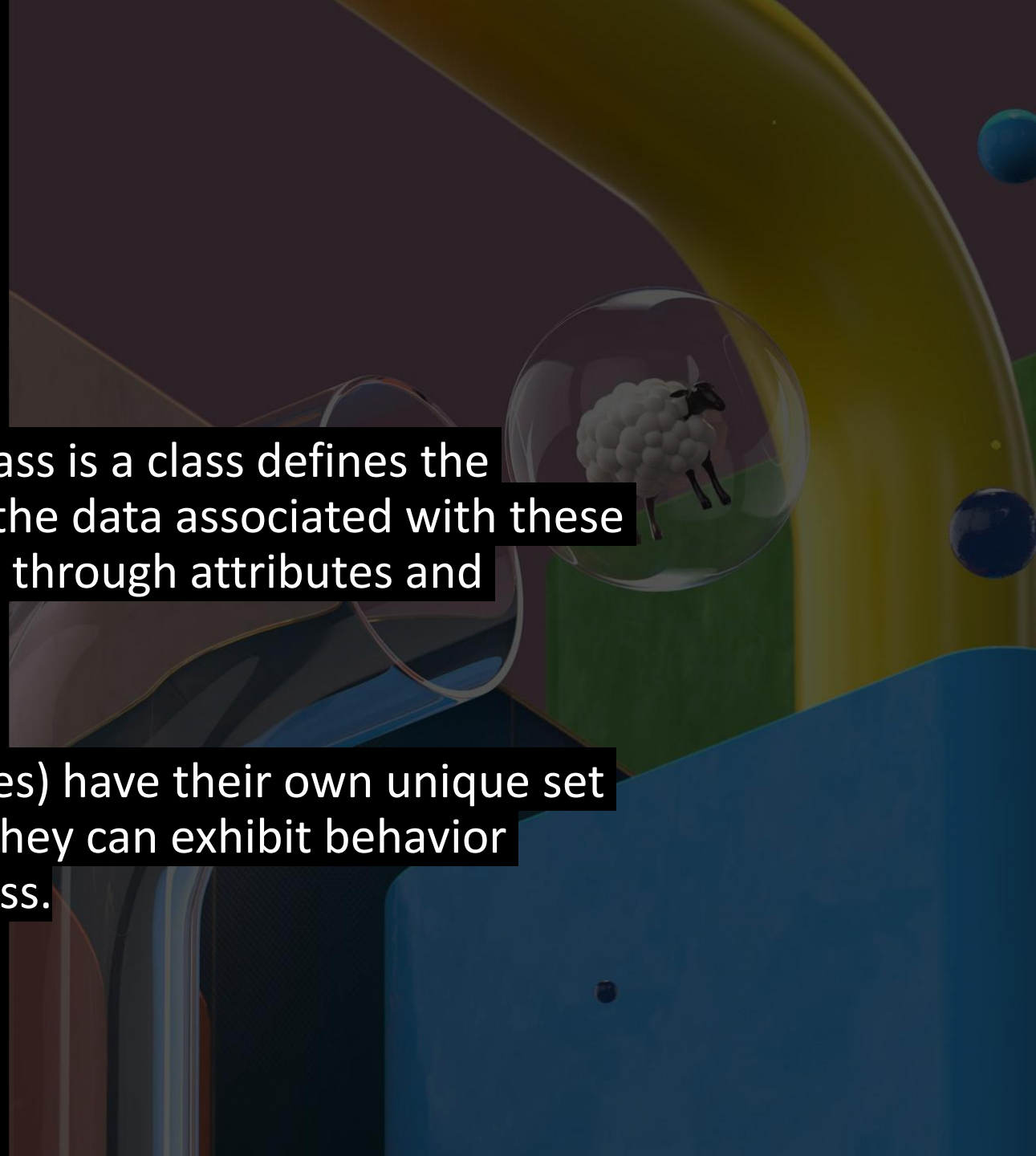
- A class defines the properties (attributes) and behaviors (methods) that its objects will have. The properties are essentially variables that hold data, and the methods are functions that perform actions on that data.



Classes and Data

- The relationship between data and a class is a class defines the structure and behavior of objects, and the data associated with these objects is encapsulated within the class through attributes and properties.

- Objects created from the class (instances) have their own unique set of data stored in these attributes, and they can exhibit behavior through the methods defined in the class.



Modular



Encapsulation: Internal details are hidden; only necessary interfaces are exposed



Separation of concerns: Each method or class does one thing well



Reusability: Modules can be reused in other contexts



Maintainability: Changes in one module don’t ripple through the system

```python
class Student:
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        self._grades = []

    def add_grade(self, grade):
        self._grades.append(grade)

    def get_grades(self):
        return self._grades

    def get_name(self):
        return self.name

    def get_id(self):
        return self.student_id
```

Correctness

- Fulfills its specification: Does what it claims to do—no more, no less
- Handles valid input properly: Produces accurate results for expected cases
- Rejects invalid input: Detects and prevents bad data or misuse
- Avoids logical errors: No off-by-one mistakes, misused variables, or broken conditions

```python
class Student:
    def __init__(self, name, student_id):  # validates name and student ID format
        if not name or not isinstance(name, str):
            raise ValueError("Name must be a non-empty string.")
        if not isinstance(student_id, int) or student_id <= 0:
            raise ValueError("Student ID must be a positive integer.")
        self.name = name
        self.student_id = student_id
        self._grades = []

    def add_grade(self, grade):  # Ensures grades are numeric and within 0-100
        if not isinstance(grade, (int, float)):
            raise TypeError("Grade must be a number.")
        if not (0 <= grade <= 100):
            raise ValueError("Grade must be between 0 and 100.")
        self._grades.append(grade)

    def get_average(self):  # returns None if no grades, avoids divide-by-zero
        if not self._grades:
            return None
        return sum(self._grades) / len(self._grades)
```

Clarity and Style

- Descriptive names: Methods and variables say what they do
- Consistent formatting: Indentation, spacing, and line breaks follow a pattern
- Logical flow: Code reads top-to-bottom like a well-organized paragraph
- Minimal nesting: Avoids deep indentation or tangled logic
- Avoids cleverness: Prioritizes clarity over showing off

```python
class Student:
    def __init__(self, name, student_id):  # reads top-to-bottom like a story
        self.name = name
        self.student_id = student_id
        self.grades = []

    def add_grade(self, grade):  # converys purpose and clean formatting
        self.grades.append(grade)

    def average_grade(self):  # conveys purpose and clean formatting
        total = sum(self.grades)
        count = len(self.grades)
        return total / count if count > 0 else None

    def summary(self):  # conveys purpose and clean formatting
        return f"{self.name} (ID: {self.student_id}) has {len(self.grades)} grades."
```

Robustness

- Validates inputs: Doesn’t assume inputs are correct
- Handles edge cases: Works even when data is missing, empty, or unusual
- Fails safely: Raises clear errors or returns fallback values instead of crashing
- Avoids silent bugs: Doesn’t ignore problems or produce misleading results

```python
class Student:
    def __init__(self, name, student_id):
        self.name = name
        self.student_id = student_id
        self.grades = []

    def add_grade(self, grade):  # checks types and range
        try:
            if not isinstance(grade, (int, float)):
                raise TypeError("Grade must be a number.")
            if not (0 <= grade <= 100):
                raise ValueError("Grade must be between 0 and 100.")
            self.grades.append(grade)
        except Exception as e:  #  fails without crashing
            print(f"Failed to add grade: {e}")

    def average_grade(self):
        if not self.grades:  # No silent bugs
            print("No grades available.")
            return None
        return sum(self.grades) / len(self.grades)
```

Scalability

- Extensible structure: Easy to add new methods or subclasses
- Minimal duplication: Reuses logic instead of copying it
- Clean interfaces: Other code can interact without knowing internal details
- Future-proofing: Anticipates likely changes (e.g., new sensor types, formats, or behaviors)

```python
class Student:
    def __init__(self, name, student_id):  # easy to sub-class
        self.name = name
        self.student_id = student_id
        self._records = {}  # flexible data model

    def add_record(self, category, value):  # add_record works for any category
        if category not in self._records:
            self._records[category] = []
        self._records[category].append(value)

    def get_records(self, category):  # no hardcoded fields
        return self._records.get(category, [])
```

Class links

Classes

https://www.w3schools.com/python/python_classes.asp

https://realpython.com/python-classes/

https://www.geeksforgeeks.org/python/python-classes-and-objects/